HOTEE: Bridging Heterogeneous Encrypted Databases with Secure and Logarithmic-complexity Search Index

Abstract

An *Encrypted Database (EDB)* utilizes either homomorphic encryption (HE) or trusted execution environment (TEE) to store, search, and process sensitive data on the public cloud. Organizations pick HE or TEE by their distinct trust, and it is highly desirable for them to share data and make joint queries across EDBs. However, EDBs with HE and EDBs with TEE are now isolated, because HE data does not share the same usability as TEE data where data can be securely decrypted to support general queries with logarithmic search complexity.

We propose HOTEE, the first framework that can support general queries on distributed heterogeneous EDBs on both TEE and HE data with logarithmic search complexity. HOTEE features a new *Merkle rangeFilter Tree* (MFT) abstraction. MFT embeds general search operations into a secure tree-based data structure that provides logarithmic search complexity in a ciphertext domain, bypassing the inherent generality and inefficiency problem caused by operation-specific encryption schemes used by prior work. Moreover, by searching on MFTs, HOTEE precisely locates and interacts with query-dependent organizations to securely transform query results cryptographically and then aggregate the results. We implemented HOTEE and applied it to two popular databases: MySQL and WiredTiger. Evaluation shows that HOTEE is the only framework that supports general queries on heterogeneous EDBs, while achieving up to 82% lower latency and up to 15.2X higher throughput compared to baselines.

1 Introduction

The prosperity of cloud computing and confidential computing fosters the deployment of *encrypted database* (EDB) on the public cloud [2, 4, 42, 48]. An EDB instance (for short, an EDB) is increasingly *heterogeneous*: typically, it simultaneously enables both homomorphic encryption (HE) and trusted execution environment (TEE) to store, search, and process sensitive data on HE hosts (i.e., nodes) and TEE nodes. For example, a ByteDance's Jeddak database instance [23] simultaneously enables Jeddak-TEE nodes to store and analyze intra-company data and Jeddak-HE nodes to process highly sensitive inter-company data. The Canada government launches an EDB project using TEE nodes to store citizens' statistics and developing HE nodes to link data dispersed across public institutions [56].

The confidential benefit of EDB makes it particularly attractive on creating a principled unified query system, to be created in this paper, for multiple parties. Specifically, those parties who require to jointly analyze sensitive data and to use

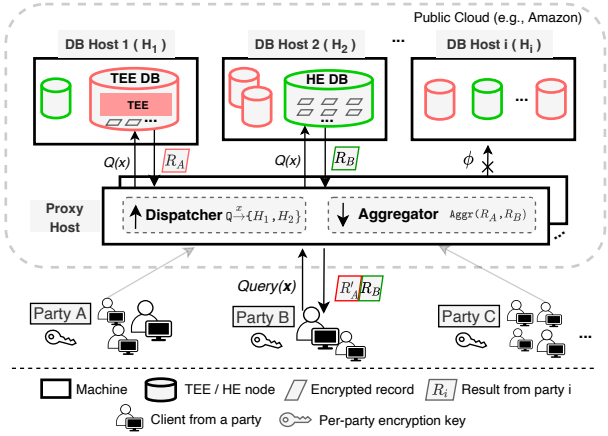


Figure 1: A principled unified query system proposed by HOTEE.

heterogeneous EDB (containing both HE and TEE nodes), as depicted in Figure 1. We give an example to show the unique benefit of such unified query system: supposing TikTok and Amazon both have an *item* table, TikTok uses TEE nodes and Amazon uses HE nodes to store their tables, and they collaborate to query both parties' encrypted data for business analytics. A TikTok salesman may submit a query as:

Select *item.ID*, *item.Price*, *item.Supplier*
From `TikTok.TEEDB.item` **Left join** `Amazon.HEDB.item`
On *range.left* $<$ *item.Price* $<$ *range.right*

The above query selects data from two mutually untrusted parties by first range searching on HE nodes and then aggregating the searched HE data with local TEE data for collaborative analytics. Thus, such a unified query system will bridge the silos between TEE and HE data and enable multiple parties to collectively answer queries on heterogeneous EDB nodes.

In order to realize the benefits of enabling secure dataflow across TEE and HE nodes, and joint query across multiple parties, the unified system we create in this paper will require a principled index component (for short, *proxy*), which should be able to efficiently dispatch general queries including both equality and range queries, to execute in relevant databases and aggregate all queried (searched) data for the clients.

Unfortunately, building such a principled unified system remains an open problem; the core difficulty lies in dispatching queries across TEE and HE nodes. Specifically, to dispatch queries to relevant TEE and HE nodes that have matching data and execute queries in these nodes, the principled index must efficiently search on both TEE and HE data. However, HE data does not share the same usability as TEE data when data is flowing across different nodes, because unlike TEE that can

decrypt data and then search plaintexts on a TEE-shielded B$^+$-tree [2, 48], HE data is fundamentallly infeasible to be decrypted and searched on plaintexts.

Despite much academic and industrial effort in developing EDBs, all exsiting EDBs (categorized into two lines of work) are infeasible to be extended to create the principled unified query system with a principled index, as they face a fundamental *generality-efficiency* dilemma identified in our paper. The first line of work (e.g., HElibDB [4], FHEDB [38]) uses specific HE function (e.g., FLT [4]) to evaluate only the equivalence of HE data by linearly scanning the whole dataset, which makes this line of work unsuitable for the goals of this paper, as it is neither general (lacking support for range queries) nor efficient (given massive and constantly growing amounts of data) on HE nodes. The other line of work (e.g., CryptDB [42], Monomi [51]) supports logarithmic complexity equality and range queries on HE data by using distinct encryption schemes (i.e., DET [12] and OPE [5]) to support either type of search operation. However, this line of work inherits the limitations of inefficient cryptographic computations and does not support dynamic changing or migrating data across the same HE nodes or TEE nodes.

This paper presents HoTEE[1], the first unified query system that can support general queries on heterogeneous EDB (i.e., simultaneously enabling HE and TEE nodes) from multiple parties with logarithmic search complexity. With HoTEE, a heterogeneous EDB instance enables both TEE nodes (e.g., EnclaveDB [44], Azure EdgelessDB [2]) and HE nodes (e.g., CryptDB [42], HElibDB [4]) to automatically deploy and connect through our proxy, which dispatches every general query to its relevant EDB nodes by securely searching on TEE and HE data. Then, heterogeneous TEE nodes and HoTEE-augmented HE nodes can execute general queries locally with logarithmic search complexity, finally all searched data are securely aggregated and sent back to the clients.

HoTEE tackles the generality-efficiency dilemma faced by all existing work via a new *confidential index* abstraction called *Merkle rangeFilter Tree* (MFT). Our principle insight is that, instead of using operation-specific encryption schemes as in prior work [4, 38, 42], we extract *deterministic* and *order-preserving*, two essential properties for realizing general queries, to create a secure and searchable data structure in a ciphertext domain. With MFT, we ensure ciphertexts in the index are deterministically encrypted and order-preserved, thus supporting both equality and range queries while the first line of work does not support range queries; by searching on MFT's tree-based data structure, we ensure logarithmic search complexity on ciphertexts, thus bypassing the inferent inefficiency problem of the second line of work that uses operation-specific encryption schemes.

To build the MFT confidential index, our technical insight is that the traditional bloom filter intrinsically holds the confidential and deterministic properties that we require to create a principled index: a bloom filter deterministically checks the existence of data objects by comparing each element in a filter while hiding the plaintexts of all objects. However, a bloom filter is not searchable, thus we break a traditional *monolithic* bloom filter into *subfilters*, where each subfilter represents only one data object. By stacking subfilters into a Merkle-tree style recursive data structure, MFT executes equality queries by traversing and comparing each subfilter node logarithmically (§4.1), while the first line of work requires linear scanning of all data. Furthermore, to support range queries, we propose *biased range* to securely patch the order-preserving property to subfilters, such that subfilters can preserve the same numerical orders as their corresponding plaintexts and search for ranges with subfilters (§4.2), while the second line of work inherits the inefficiency brought by using specific encryption schemes for range queries.

We implemented HoTEE on the CryptDB codebase [42], a mature modular framework for evaluating distributed encrypted databases. We built two HE databases with HoTEE by augmenting prominent databases: HoTEE-MySQL and HoTEE-WiredTiger. MySQL [39] is the most widely used SQL backend for encrypted databases; WiredTiger [16] is a popular key-value store that is the default NoSQL backend for MongoDB. We run HE database with an open-source TEE database Azure EdgelessDB [2] for joint query, and we design a secure data aggregation protocol (SDA) to cryptographically transform and aggregate searched multi-party data from heterogeneous EDB nodes (§4.3). We compared HoTEE with two famous HE databases CryptDB [42] and HElibDB [4] using typical workloads (e.g., TPC-C [34]) evaluated in relevant systems [8, 42, 55]. Evaluation shows that:

- **Generality**. HoTEE is the first unified query system that supports general queries (equality, range, join, insert, update, and delete) among heterogeneous HE and TEE nodes within the same EDB instance (§6.1), while the first line of work supports only equality queries; and HoTEE supports logarithmic search complexity on all these queries, proved in §4.1 and empirically evaluated in §6.2: when the size of dataset expanded, HoTEE's latency increased logarithmically.
- **Efficiency**. Although CryptDB supports general queries, HoTEE achieved up to 28% lower latency and 1.8X higher throughput than CryptDB (§6.1). Although HElibDB supports dynamic changing data on HE nodes, which CryptDB does not support, HoTEE achieved up to 93.8% lower latency and 15.2X higher throughput than HElibDB (§6.1). Meanwhile, HoTEE is the first work to support multi-party joint query on heterogeneous EDB containing both TEE and HE nodes, compared to all baselines.
- **Extensibility**. HoTEE supports a SQL (MySQL) instance consisting of both HE nodes and TEE nodes; and it supports another NoSQL (WiredTiger) instance consisting of both HE nodes and TEE nodes.

Our main contribution is MFT, a new confidential index

[1] HoTEE stands for Homomorphic-TEE joint query.

	HoTEE	MySQL [39]	EnclaveDB [44]	EdgelessDB [2]	Cryptdb [42]	HElibDB [27]	FHEDB [38]	HybrTC [54]	Dory [20]
Confidentiality	✓	✗	✓	✓	✓	✓	✓	✓	✓
Joint query	✓	✗	✗	✗	✗	✗	✗	✓°	✓°
Equality search	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	$O(n)$	n/a.	$O(n)$
Range search	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	n/a.	n/a.	n/a.	n/a.
Insertion	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(1)$	$O(1)$	$O(1)$	n/a.	$O(1)$

°: Potentially support, but not explicitly handle joint queries to heterogeneous EDBs.

Table 1: Comparison of HoTEE and existing systems. "**n/a**." means not applicable as they do not support such query operations.

abstraction to enable general queries with logarithmic search complexity in a unified EDB instance, containing both HE and TEE nodes. A long-lasting open problem in the EDB area is that, existing EDBs support only limited queries on either TEE or HE, HoTEE efficiently bridges heterogeneous EDB and is the first work that supports multi-party joint query in each EDB instance. Our other contributions include a secure data aggregation protocol, which is the first effective attempt to enable multi-party data aggregation in each heterogeneous EDB, making HoTEE unique to facilitate multiple parties with distinct trust within the same EDB instance to securely share data and make joint queries in various mission-critical applications (e.g., medical diagnosis [33, 35], governmental data collection [11, 56]). HoTEE's source code is released on `github.com/osdip281/HoTEE`.

2 Background and Motivation

2.1 Securing Data for Cloud Databases

Today, organizations are increasingly moving their constantly growing amounts of data to cloud databases [1, 3]. Yet, many organizations still decline to widely adopt cloud databases for data management due to severe confidentiality concerns. To address the security issue, techniques have emerged that allow queries to be run against encrypted data kept in cloud databases. Two prominent approaches consist of hardware-based solutions using *Trusted Execution Environment* (TEE) like Intel SGX [17] and software-based solutions using *Homomorphic Encryption* (HE) like BGV [24].

Intel SGX. SGX [17] is a pervasively used hardware feature on commodity CPUs. SGX provides a secure execution environment called *enclave*, where data and code execution cannot be seen or tampered with from outside. SGX has been widely adopted in recent encrypted databases [2, 44, 48] because (1) SGX provides sophisticated API including sealing and attestation, and (2) the latest SGX 2.0 greatly expands enclave memory capacity (up to 512GB/CPU [30]) to fit memory-intensive applications including databases [48].

Homomorphic BGV. To avoid trusting one TEE hardware vendor, some organizations are seeking a cryptographic root-of-trust from homomorphic encryption (HE), and are willing to pay for the deficiencies of using HE-based solutions, especially fully HE schemes like BGV [24]. BGV allows arbitrary aggregation (e.g., `sum`, `avg`) on encrypted data (with noises) without requiring a secret key for decryption, and the computation result can be fully revealed by the owner of the secret key [24]. BGV uses *key switch* algorithm to reduce ciphertext's noises, which is leveraged by HoTEE to aggregate multi-party data encrypted by distinct secret keys (§4.3).

2.2 Heterogeneous Encrypted Databases

To make a clear comparison, we summarize the representative systems and compile their features in Table 1. Note that, the traditional MySQL database uses access control and data-at-rest encryption to protect data [40], failing to provide data and query confidentiality (i.e., process encrypted queries on the encrypted data to defend against adversaries described in §1), but enjoys full-fledged search functionalities with logarithmic-complexity indexing on plaintexts using B^+-tree [39].

TEE databases. The first category is TEE-based encrypted databases. EnclaveDB [44] makes user data completely inaccessible by implementing in-memory storage and query engine in enclaves, protecting data and query confidentiality and supporting logarithmic-complexity operations using a TEE-shielded B^+-tree. Azure EdgelessDB [2], and Alibaba Enclage [48] consider on-disk storage (sealed by SGX) and use B^+-tree for efficiently indexing as well: Enclage shields most B^+-tree nodes in plaintext within enclaves; EdgelessDB stores an AES-encrypted B^+-tree outside enclaves, and performs searches by on-demand pulling index nodes to enclaves and then decrypting and comparing each node for indexing.

HE databases. Different from TEE databases that decrypt and then search plaintexts on a fully (or partially) shielded B^+-tree in enclaves, the homomorphic data is forbidden to decrypt and search in enclaves because HE database users demand a cryptographic root-of-trust thus distrust TEEs (§2.1).

To build search indexes for HE databases, CryptDB [42] uses deterministic encryption [12] for equality search by linearly scanning all data and uses order-preserving encryption [5] (OPE) for range search. Since OPE can preserve the original orders of plaintexts, one can use off-the-shelf indexes like B^+-tree for OPE ciphertexts same as plaintexts. However, OPE does not support equality search, and CryptDB's solution (i.e., onion encryption) becomes inefficient when the workload is mixed [51]. Besides, OPE assumes the plaintext domain distribution is fixed [37], which contradicts the dynamic nature of database writes. HElibDB [4] and FHEDB [38] use specific cryptographic tools to directly perform equality searches on homomorphic data with linear-complexity but lack support for range search on homomorphic data.

Hybrid-trust joint query. The aforementioned systems are designed for a single machine and do not support distributed

queries on multiple hybrid-trust databases. HybrTC [54] integrates TEE with MPC [18] to support multi-party data analytics wherein users only trust their own TEE databases, but supports neither equality nor search queries on HE databases. Dory [20] supports only equality search for encrypted keywords by performing linear scans on bloom filters [13] because the bloom filter has limited expressiveness which tells only the existance of searched data on an encrypted database.

2.3 Motivation and Opportunities

From the survey results reported above, we summarize that different organizations can have different trusts and preferences in encrypted databases. Thus, in this paper, we envision the first hybrid-trust scenario where users make distributed queries to heterogeneous encrypted databases. Such a computing model brings two unique challenges. First, HoTEE should support general query operations with logarithmic search complexity on HE databases for joint query. Second, HoTEE should support encrypted query filtering and multi-party data aggregation same as distributed plaintext databases.

To tackle the challenges, we summarize what we considered opportunities to make the hybrid-trust joint query problem easier. First, we observe that filters used by traditional databases have an intrinsic stealth feature of hiding plaintexts while providing membership information, which can serve as a basic block to build an encrypted search index. Second, we can employ traditional homomorphic key switch algorithm to securely aggregate multi-party data, even if the algorithm has an orthogonal goal of reducing homomorphic data noises.

3 Overview

3.1 System Model

Entities. As shown in Figure 1, HoTEE has three roles: *host*, *proxy*, and *client*. Multiple clients (e.g., physicians) form a single party (e.g., hospital), and multiple parties outsource their encrypted data to database instances (for short, database) run on public cloud hosts for encrypted query service. Same as plaintext distributed databases [15, 43], clients first submit queries to a proxy, which profiles encrypted queries, filters query conditions, and then re-directs queries to relevant databases to retrieve encrypted data. The retrieved data is aggregated at the proxy and then returned to clients.

Data model. Without losing generality, we use the relational data model as an example to illustrate our design in the following discussions. We will discuss the extensibility in §6.5.

Specifically, a table T contains n columns (or attributes) $CL = \{c_0, c_1, ..., c_{n-1}\}$ and has a primary key $PK \in CL$. Same as [44,48], we consider the values in all columns are confidential, which means that each value should be encrypted before being sent to a database. For a record (i.e., a row) r, its value v_i in column c_i is ciphertext $E(v_i)$, where E represents the encryption scheme employed by a party: for TEE databases, E is typically AES-GCM [32] and the party's encryption key will

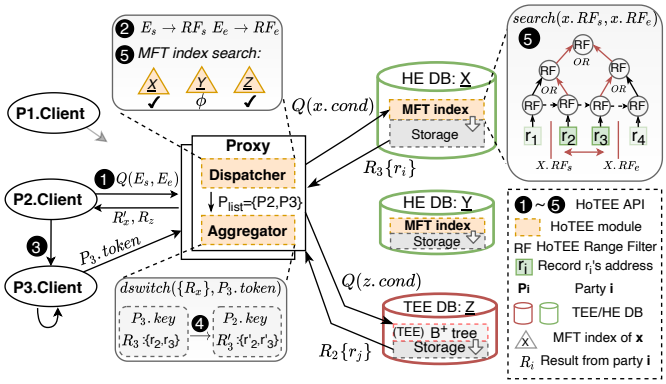


Figure 2: HoTEE's architecture, details described in §3.3.

be provisioned to SGX enclaves when computation over plaintext is needed; for HE databases, E is typically BGV [24], and the party never reveals its BGV encryption key to databases or other parties participated in joint queries.

Note that, same as existing encrypted databases [2, 44, 48], columns can be encrypted by distinct encryption keys to further strengthen confidentiality; it is also feasible to have plaintext columns if cross-column correlations are omitted [9, 10].

Storage model. Same as existing work [20, 48], in HoTEE, we focus only on the search system for end-to-end encrypted databases and not on the design of underlying storage engine. Many optimized storage engines [26, 47, 48] already exist and are used in encrypted databases. HoTEE is built on top of these storage engines, so below we abstract out functions that should be supported by an underlying storage engine.

Recall the aforementioned data model, we assume a table T with n columns $CL = \{c_0, c_1, ..., c_{n-1}\}$ and is indexed on the primary key c_0 (i.e., $PK = c_0$). A record r can be depicted as $[E(k), E(v_1), ..., E(v_{n-1})]$ where k is the value for c_0. The storage engine should provide the following API:

- EqualGet(E(k)): Retrieve the row r from T given a key k.
- RangeGet(E(k_s),E(k_e)): Retrieve all rows $\{r\}$ from T whose keys are between E(k_s) and E(k_e).
- FullGet(): Retrieve all rows $\{r\}$ from T.
- Put(r): Insert (or update) a row r to T if its key k is non-existent (or existent).
- Delete(E(k)): Delete the row r from T given a key k.

Note that, same as [2,42,48], HoTEE also supports searching on non-primary keys by building secondary indexes [46].

3.2 Threat Model

HoTEE focuses on protecting the end-to-end confidentiality of all parties' data stored at database hosts. All roles in HoTEE are considered to be semi-honest. Specifically, same as existing single-party encrypted databases [4, 38, 42], we consider (1) a semi-honest database host which follows HoTEE's protocols but passively observes stored data, index items, and query results to deduce plaintexts, and (2) a semi-honest proxy which is curious about clients' queried data and query results.

LIBHOTEE API	Role	Description
DATA PROCESSING API		
❶ `newQry(COND)`	CL[1]	Submit a new query with encrypted conditions (e.g., $E(k)$) to HOTEE's proxy (§4.2).
❷ `transform(E(K))`	PR[2]	Transform a key k to HOTEE's searchable index key (§4.2).
❸ `reqTok(KS)`	CL	Request new tokens from other clients for key switch (§4.3).
❹ `dSwitch(TK,D)`	PR	Run distributed key switch protocol to aggregate multi-party encrypted data (§4.3).
INDEXING API		
❺ `MFTSearch(COND)`	CL, DB[3]	Read/write by searching on HOTEE's MFT index (§4.2).

[1] **CL**: CLient [2] **PR**: PRoxy [3] **DB**: DataBase instance

Table 2: The API provided by LIBHOTEE.

Similar to existing multi-party databases [20, 54], we consider all parties to be semi-honest, which means that (1) the outsourced data is correct, and queries from clients are authenticated through out-of-band services like [52], and (2) different parties have distinct trust in TEE or HE, thus we consider TEE data can be securely decrypted and processed in TEE's secure memory space (e.g., SGX enclave [30]) while HE data can never be decrypted to plaintexts.

Same as [2, 8, 42, 48], we do not protect metadata security. Specifically, HOTEE does not hide numerical orders between two ciphertexts, since existing oblivious approaches can lead to impractical performance penalty [19, 22]. Also, protecting coarse statistical properties such as the names of columns, the volume of indexes, or the length of values, is out of our scope. Same as [20, 42], we assume the underlying storage engine (§3.1) is maliciously secure, which can always retrieve the correct data requested by hosts. Providing such a guarantee (e.g., detecting rollback and fork attacks) has being well-studied in many storage systems [14, 50].

3.3 HOTEE's Workflow Overview

Figure 2 shows HOTEE's workflow with three phases.

• **Phase 1: query dispatch.** Typically, distributed databases use a proxy to dispatch client's queries to relevant database instances (for short, databases) for cost-efficiency. In HOTEE, clients submit signed queries (❶) to the proxy via a TLS-enabled link. The proxy will drop the connection if the client sends malformed queries (e.g., with invalid signatures from unknown parties) to avoid the client mouting DoS attacks on the proxy. For performance, queries are encrypted via AES, which are implemented efficiently in hardware.

We let HOTEE's proxy run a dispatching thread to determine which encrypted databases should be forwarded to. In Figure 2, we use an example of range query Q to show the workflow. The proxy decrypts range conditions E_s and E_e, transforms them into HOTEE's confidential index primitive *range filter*, denoted as RF_s and RF_e (❷). These two range filters serve as the identifiers of ciphertexts, which are *deterministic* and *order-preserving*, hence can be leveraged for equlity search and range search without revealing plaintexts (§4.2). HOTEE searches RF_s and RF_e in pre-stored *MFT* indexes (introduced in **Phase 2**) for each database (❺), forwards Q to a database d if the searching result on d's MFT index is not ϕ, and generates a list of parties P_{list} that Q requested, to the aggregating thread (introduced in **Phase 3**).

• **Phase 2: local search.** Queries are forwarded to TEE or HE databases for local encrypted data searching, and both encrypted databases follow the separation of storage and unclustered indexes same as [48], so that HOTEE can inherit the mature safety and performance optimizations from existing storage systems [50]. Upon receiving Q, a TEE database decrypts query conditions in enclave, searches plaintext conditions in a TEE-shielding B^+-tree (red dotted box) where each tree node's value is plaintext, and calls the storage API (§3.1) to retrieve data. The on-plaintext B^+-tree searching brings TEE databases a natural advantage of supporting a full set of query operations, including *equality*, *range*, and *aggregation*.

For HE databases, different from existing systems that use specific cryptographic functions to support designated query operations with high-complexity (§2.2), HOTEE searches RF_s and RF_e on *merkle range filter tree* (MFT), a new confidential index that bridges the generality gap with TEE databases (❺). MFT is build on top of HOTEE's index primitive *range filter* (RF), by stacking RFs into a merkle-tree style structure where each tree node's value is the *OR* gate computation result of its child nodes. In MFT, leaf nodes (nodes in the deepest tree layer) point to storage addresses of corresponding records, and all leaf nodes are linked for fast data retrieval same as traditional B^+-tree (§4.2).

• **Phase 3: data aggregate. Phase 3** runs in parallel with **Phase 2**. In **Phase 3**, an aggregating thread running at the proxy executes HOTEE's distributed key switch protocol to securely convert the retrieved encrypted data from multiple parties into client-decryptable data for usability (§4.3). In the example of Figure 2, the databases' searched results contain R_2 from party P_2, and R_3 from party P_3. Since Q is submitted by P_2's client, the aggregating thread fetches a secure token from P_3 (❸); upon receiving R_3, to ensure data condidentiality, our protocol securely converts R_3 into P_2's encryption key $P_2.key$ using that token, without having to decrypt R_3 (❹). By doing so, clients from different parties can securely decrypt retrieved data from other parties without directly sharing their sensitive encryption keys.

3.4 The HOTEE API

HOTEE provides a library called LIBHOTEE, which contains basic API (Table 2) that facilitates heterogeneous encrypted databases to be deployed in HOTEE to jointly answer general queries. We will discuss how to integrate an encrypted database into HOTEE framework using LIBHOTEE in §4.2. The data processing API consists of functions for clients to submit general queries and for proxy and databases to process

encrypted queries (§4.3). The indexing API enables the proxy and databases to search encrypted data with HoTEE's confidential index: if called by the proxy, `MFTSearch(k)` returns *True* if the key k exists in a database; if called by the host, `MFTSearch(k)` returns the address of k in storage and calls the storage API (§3.1) to retrieve local data (§4.2).

Why is LibHoTEE essential? The LibHoTEE API poses two bare essentials that cannot be replaced by manual customization. Firstly, by using the API, HoTEE automates the process of deploying, searching, and updating confidential indexes on encrypted databases (§4.2), empowering general queries with logarithmic search complexity and facilitating multiple parties to securely aggregate searched encrypted data (§4.3). Secondly, the API gives security hints to clients at runtime by detecting manipulation or omission of query results based on HoTEE's MFT confidential index (§4.2).

Overall, the highlight of HoTEE's workflow stems from achieving generality and low complexity on hybrid encrypted databases in semi-honest environments, and we illustrate in two aspects. Firstly, HoTEE's new MFT index tackles a full set of query operations (equality, range, and aggregation) on encrypted data, and achieves $O(\log n)$ time complexity (n is the total number of records) on equality and range search. Since aggregation is not a search operation on indexes, its complexity depends on the used algorithm (e.g., BGV [24] in HE databases). Besides, MFT can be generalized to support NoSQL databases and general types of attribute (e.g., real-values and string, see Appendix A.2). Secondly, different from existing single-party encrypted databases [42, 48], HoTEE is the first framework to securely aggregate multi-party encrypted data without revealing decryption keys in a semi-honest environment (§4.3), making HoTEE unique to facilitate multiple parties share sensitive data and jointly answer queries.

4 Protocol Description

This section first describes a strawman index design inspired from bloom filter (§4.1) and then introduces HoTEE's new MFT index based on the strawman index (§4.2). Then we present *SDA*, a secure data aggregation protocol that locates query-dependent parties using MFT, transforms and aggregates multi-party data into querier-decryptable data (§4.3).

4.1 A Preliminary Subfilter-based Index

Preliminaries. A bloom filter [13] is a space-efficient data structure that provides membership information: all elements are added to a single set and it checks whether an element is a member of a set. It also has an intrinsic *confidential* characteristic: the elements themselves are not added to a set, instead the hash is added to the set. Formally, a bloom filter B is an array of m bits and parameterized by k independently uniform hash functions $\{H_k\}$, where $\{H_k\}$ map an element to k positions in B uniformly, and all elements are mapped to a single array in a *monolithic* manner.

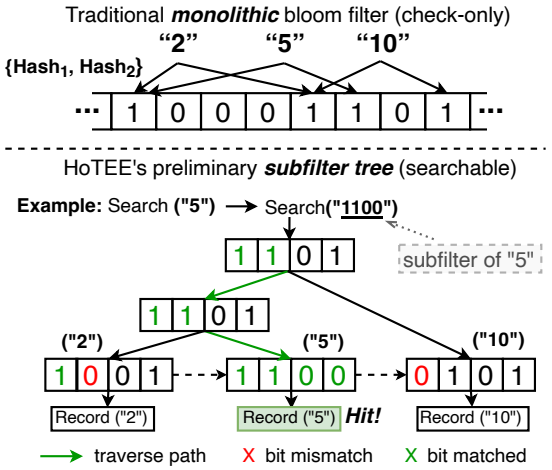


Figure 3: An initial search index built on top of subfilters.

Traditional monolithic bloom filter supports confidential membership check. Specifically, to add an element x to B before checking, one uses the hash functions in $\{H_k\}$ to hash x and sets all bits at the position of hash values to `1`. Then, to check whether x is in the set, one also hashes x using the k hash functions. If any bit at the position of hash values is 0, x is definitely not in this set (none false negatives); otherwise, y is in the set (with small configurable false postives).

Overall, traditional monolithic bloom filter supports membership check using bit representation and inherits the confidentiality guarantee as long as we make standard assumptions on the one-wayness of cryptographic hash functions [25],

Subfilter-based search index. Owing to the monolithic design, a traditional bloom filter supports only membership check by telling whether an element (hash) is in a set, nevertheless, it cannot serve as an index for locating data in storage.

We observe that the confidential characteristic of bloom filters can be useful for encrypted search, thus in our initial design, we divide a monolithic bloom filter into *subfilters* where each subfilter stores the hash (in bits) for only *one* element. As shown in Figure 3, subfilters have a constant array size and are recursively stacked in a Merkle-tree style for encrypted search. Specifically, each leaf node in the subfilter tree stores one subfilter of an index key (e.g., `"1001"` for index key *2*) and sorted in plaintext key order; for each inner node (all nodes except the last layer), its value is the element-wise *OR* gate computation result of its child nodes' subfilters: given an m length subfilter, for an inner node *inner* with k child nodes cn_k, the inner node's subfilter is computed as:

$$inner[i] = cn_1[i] \; or \; ... \; cn_t[i] \; or \; ... \; cn_k[i] \; (\forall i \in [0,m]) \quad (1)$$

Same as B$^+$-tree, only the leaf nodes store values (e.g., record pointers) and inner nodes are only for indexing (tree traversal).

The design rationale behind subfilter tree is that, if a `"1"` bit is present in a child node's subfilter, after *OR* computation with other child nodes' subfilters, its parent node's subfilter must contain that `"1"` bit (e.g., if $cn_t[0]$ =`"1"`, it always holds $inner[0]$ =`"1"`). This implies HoTEE's new insight of using

existence information for searching: if one element exists in a subfilter node (i.e., hashes of the element are set to "1"), the element must exist in its parent node's subfilter. This enables HoTEE to traverse a subfilter tree in a top-down manner by verifying the existence of query parameters in subfilter nodes with confidentiality, rather than compare the plaintext values as in traditional B$^+$-tree.

Based on the subfilter tree, to conduct an equality search, instead of searching with plaintext query parameters (for short, parameter), the client generates an encrypted subfilter of the parameter using Equation 1, then the database traverses the subfilter tree to locate the corresponding record of the parameter (Figure 3). Specifically, each inner node conducts *subfilter comparison* to verify whether every "1" bit of the parameter exists in the current node's subfilter. If so, the parameter will be passed to the child nodes iteratively untill reaching the bottom layer. When reaching the leaf nodes, different from the verification in inner nodes, a leaf node must verify that every single bit in its subfilter matches the parameter as one subfilter comprises only one index key.

The equality search strategy can be extended to support range search. Given two range parameters (i.e., left and right boundaries), they are first transformed to two subfilters using Equation 1, then the subfilter tree searches two subfilters same as the equality search and locates two leaf nodes (if matched). Last, since leaf nodes are sorted during index construction, all leaf nodes between the two searched leaf nodes are returned as searched results same as B$^+$-tree.

4.2 HoTEE's Complete Confidential Index

Limitation of strawman subfilter index. Although the subfilter tree (§4.1) can support equality and range query with logarithmic search complexity, however, since the subfilter tree is driven by existence verification rather than numerical comparison as in traditional B$^+$-tree and there is no *order* among subfilters, two fundamental limitations (**L1, L2**) arise.

First (**L1**), a subfilter tree cannot support non-existent key search. Apparently, range queries can search on parameters that are non-existent in databases as clients can choose whatever search boundaries they want. This is not a problem in traditional B$^+$-tree because index nodes and query parameters are plaintexts, thus the position of non-existent keys can be trivially located by finding two adjacent plaintexts. However, since subfilter is encrypted and there are no numerical orders between subfilters, a subfilter tree cannot locate positions of non-existent keys through subfilter comparison. Second (**L2**), owing to the same reason, a subfilter tree does not support write operations (e.g., insert), because the tree cannot locate the write position among sorted leaf nodes.

Design rationale. To tackle **L1** and **L2**, our key idea is that, instead of handling specific query operations using special encryption algorithms as in previous work [4,38,42], HoTEE instead embeds the query operations into a secure data structure, and the data structure is built on traditional encryption schemes that provide confidentiality guarantees.

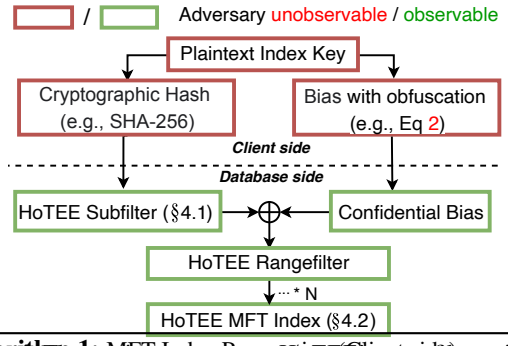


Figure 4: The routine to construct HoTEE's rangefilter primitive.

Algorithm 1: MFT Index Preparation (Client side)

```
Input:
    plaintext[]                    ▷ Plaintext keys for indexing
    chashfunc[]    ▷ Cryptographic hash functions for filters
    k, γ                           ▷ Obfuscation parameters
Output:
    rf[]      ▷ Rangefilters of plaintexts for encrypted search
Function Prepare() do
    sortedPtx[] ← sort (plaintext)
    ▽ Generate subfilters
    foreach plaintext p in sortedPtx do
        subfilter[m] ← {0}        ▷ m is subfilter's bit length
        foreach evaluator e in chashfunc do
            subfilter.set (e(p), 1)
        rf.insert (subfilter)
    ▽ Generate biases in plaintext order
    range_min ← γ, range_max ← m + γ
    start ← 0
    foreach plaintext p in sortedPtx do
        bias ← k * p + hash(p)              ▷ hash(p) ∈ [0,k)
        p.rmin ← bias + range_min
        p.rmax ← bias + range_max
        rf[start++].rng ← {p.rmin, p.rmax}
    return rf
```

Specifically, we identify *determinism* and *order*, two essential properties for equality and range search that must be retained in one searchable data structure. Recall that HoTEE's initial subfilter tree already constructs a searchable data structure with determinism by creating subfilters with deterministic plaintext-ciphertext maps, and confidentiality that directly inherits from cryptographic hash functions in bloom filters. Thus, adding orders to subfilters without sacrificing confidentiality is on the critical path of a secure index design.

Rangefilter primitive. To this end, we propose *rangefilter*, a primitive for constructing confidential search index by tackling two limitations mentioned above. The routine to construct rangefilter is shown in Figure 4. Specifically, we generate a *confidential bias* for each subfilter with a bias function that fulfils three rules. First, the function should be monolithic, ensuring that a larger plaintext always maps to a larger bias to keep encrypted data ordered. Second, to reduce the overhead at the client side, the function should be stateless, which means that no additional states (e.g., previous biases) should

be maintained at runtime, as this generation process happens on the client side. Last, the function should provide randomness such that an adversary that observes a bias should not immediately deduce the plaintext.

Definition 1 (Confidential bias function). Given two plaintexts $p_1 > p_2$, if a bias function f satisfies: (1) $f(p_1) > f(p_2)$, (2) f is independent of any previous states, and (3) f varies within a certain range of values, then f is regarded as a qualified confidential bias function for rangefilters.

We formulate that a bias function f should have the form of $f(x)=g(x)+t(x,y,z)$, where $g(x)$ is monotonically increasing on x, $t(x,y,z)$ is a parametric function which varies within a certain range and has a fixed number of values, and does not overwhelm the monotonicity of $g(x)$. Any functions that fulfill the above formulation can be applied to generate biases for subfilters. In HoTEE, we use the bias function

$$f(x) = kx + hash(x),\ hash(x) \in [0,k) \qquad (2)$$

in which x is the plaintext, k is a confidential coin, both are owned by the client. By applying bias b to a m length subfilter (i.e., m bits), a rangefilter has a *biased range* of $[b+\gamma, m+b+\gamma]$, in which γ is a random parameter generated by the client to obfuscate adversaries.

Definition 2 (HoTEE rangefilter). A rangefilter is a secure data structure comprises a subfilter (Equation 1) and a biased range (Equation 2), providing data existence and order information for encrypted equality and range search, respectively.

To remark, we emphasize that the entire process of generating rangefilters at the client side is stateless because (1) the bias function is stateless (Equation 2), and (2) generating subfilters requires only to store parameters of a fixed set of hash functions. Overall, to support encrypted query service, unlike previous work that requires clients to store stale states or even a full replica of outsourced data [41], HoTEE supports lightweight clients without paying additional storage overhead.

MFT confidential index. Based on rangefilter, we construct Merkle Rangefilter Tree (MFT), a confidential index for supporting general queries on encrypted data with logarithmic search complexity. We present the construction and search operations on MFT, and using MFT for query authentication.

• **MFT construction.** To generate rangefilters, the client takes plaintexts of index keys as input and outputs rangefilters by generating subfilters and biases respectively (**Algo** 1). Specifically, the plaintexts are sorted to ensure the encrypted data preserve the same numerical order as plaintexts (line 9). Then, subfilters are constructed using client-chosen (unknown to database) cryptographic hash functions (e.g., SHA-256) with Equation 1 (line 10∼14). Last, biases are generated using Equation 2, and are added with obfuscation parameters to produce biased ranges of subfilters, which contain order information for searching (line 15∼20).

With rangefilters, the client constructs MFT before uploading it index to the cloud database, as shown in **Algo** 2. Concretely, the client creates leaf nodes (`MFTNode`) with rangefilters built on index keys and ciphertexts (either TEE or HE data), and creates inner nodes (line 9∼13).

Algorithm 2: MFT Index Construction (Client side)

```
1  Struct {
2      Rangefilter myFilter                    ▷ Input from Algo 1
3      Ctx value                               ▷ TEE or HE ciphertext
4      Node pn                                 ▷ Parent node
5  } MFTNode;
6  Function CreateLeaf() do
       ▽ Initialize bottom-layer MFTNodes
7      ...
8      return leaf[ ]
9  Function CreateInner(Node_i, Node_j) do
10     (inner.left, inner.right) ← (Node_i, Node_j)
11     (Node_i.pn, Node_j.pn) ← inner
       ▽ Union of child nodes' ranges
12     inner.rng ← Node_i.rng ∪ Node_j.rng
13     return inner
14 Function CreatMFTIndex() do
15     le[]← CreateLeaf(), in[]← ∅, ptr ← 0
       ▽ Pair-wise inner node construction
16     while ptr < le.length do
17         if ptr = le.length − 1 then
18             in.insert(CreateInner (le[ptr],le[ptr-1].pn))
19         else
20             in.insert(CreateInner (le[ptr],le[ptr+1]))
21         ptr ← ptr + 2
       ▽ Bottom-up MFT construction
22     while in.length > 2 do
23         n[]← ∅, ptr ← 0
24         while ptr < in.length do
25             if ptr = in.length − 1 then
26                 n.insert(CreateInner (in[ptr],in[ptr-1].pn))
27             else
28                 n.insert(CreateInner (in[ptr],in[ptr+1]))
29             ptr ← ptr + 2
30         in ← n
       ▽ Entry of MFT confidential index
31     return root ← CreateInner(in[0], in[1])
```

MFT inherits two advantages of traditional B$^+$-tree. First, all MFT leaf nodes are sorted in plaintext order and linked for fast data fetching. Second, only MFT leaf nodes contain values (i.e., storage pointers of records), and other inner nodes are only for indexing. Nevertheless, different from traditional B$^+$-tree, MFT is constructed by recursively stacking rangefilters (line 14∼31), which display as an encrypted form of plaintexts that comprises HoTEE's unique subfilter and biased range.

Note that, **Algo** 2 constructs MFT index statically, however, a client can also insert, update, or delete data through MFT at runtime with search operations introduced below.

• **MFT search operations.** In MFT, the high-level idea of encrypted search is to use the biased range (for short, range) for indexing (tree traversal) among inner nodes as it preserves the numerical orders of encrypted data as in plaintexts, and use the subfilter for point match among leaf nodes as it contains the existence information of plaintexts (**Algo** 3). Since most database read/write queries are composed by equality and

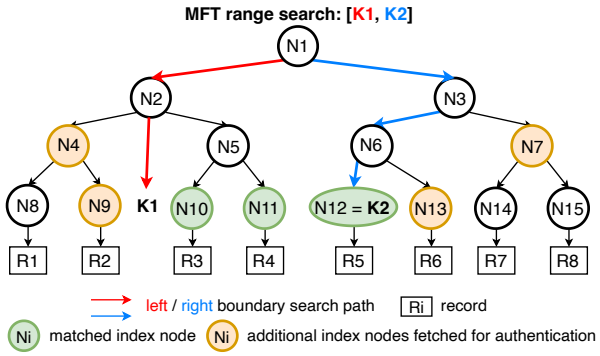


Figure 5: A typical MFT range search example on HOTEE.

range operations [36], we describe how HOTEE's MFT index handles encrypted equality and range search below.

To handle encrypted equality search, MFT compares parameter's range with each tree node's range untill the bottom layer. For each comparison, if the target's range is within the range of the tree node, it implies that the tree node is on the target's search path, because a parent node's range is the union computation result of its child nodes' ranges (**Algo** 2, line 12). When reaching the bottom layer, HOTEE compares subfilters to see if every bit in parameter's subfilter matches a leaf node's subfilter, and only when every single bit matches, the final result is searched (line 5∼14). This is because, different from a monolithic bloom filter design, HOTEE maintains one-to-one maps between subfilters and plaintexts (§4.1).

To handle encrypted range search, MFT conducts two equality searches on the left and right boundary parameters, and outputs all leaf nodes between two searched leaf nodes (line 25∼32). Besides, MFT tackles non-existent key search for parameters as shown in Figure 5. Concretely, when the left boundary is non-existent, i.e., the range of K_1 is within the range of node N_2 but none of N_2's child nodes' ranges fully contain K_1's range (owing to the monotonicity in Equation 2), MFT searches the smallest data larger than the left boundary and confirms N_{10} as the first searched index node (line 15∼19). Similarly, if the right boundary is non-existent, MFT searches the largest data smaller than the right boundary (line 20∼24). In Figure 5's case, the right boundary is existent because MFT conducts subfilter comparison at leaf nodes and finds out that N_{12}'s subfilter exactly matches K_2's. Last, all leaf nodes' records in green box (R_3∼R_5) are returned to the client (i.e., querier).

• **MFT query authentication.** As aforementioned, MFT's recursive Merkle-tree style structure enables general range queries even when query parameters are non-existent, and such structure also allows clients to efficiently authenticate queries by checking the absence and presence of data records in an outsourced database: any manipulation or omission of the query results will fail to regenerate the root subfilter, and thus be detected. To authenticate encrypted range queries, HOTEE takes the following two steps.

First (**Proof fetching**), on receiving a range query, the database searches for all records whose key falls within the search range (**Algo** 3) and constructs a proof including (1) one encrypted record to the immediate left of the lower-bound (R_l) and one encryped record to the immediate right of the upper-bound of query result (R_r), and (2) additional subfilters to assist recompute the root's subfilter, i.e. subfilters of all left sibling nodes of MFT left traversal path and right sibling nodes of the right MFT traversal path.

Algorithm 3: MFT Confidential Search (Database side)

```
1  Input:
2    root                          ▷ Entry of MFT from Algo 2
3    key (i.e., rangefilter)       ▷ Search target
4    opt (P | L | R)   ▷ Search option: Point search or range
       search with Left or Right boundary; default is P
5  Function EqualitySearch(root, key, opt=p) do
       ▽ Compare subfilters (§4.1) at leaf nodes
6      if isLeaf(root) then
           ▽ Every subfilter bit must match
7          if key.subfilter = root.subfilter then
8              return root
9          else
10             return null
       ▽ Use biased range to traverse MFT
11     if key.rng ∈ root.left.rng then
12         return EqualitySearch(root.left, key)
13     if key.rng ∈ root.right.rng then
14         return EqualitySearch(root.right, key)
       ▽ If left boundary is non-existent (Fig 5)
15     if opt = l then
           ▽ Find smallest data larger than key
16         root ← root.right
17         while !isLeaf(root) do
18             root ← root.left
19         return root
       ▽ If right boundary is non-existent
20     if opt = r then
           ▽ Find largest data smaller than key
21         root ← root.left
22         while !isLeaf(root) do
23             root ← root.right
24         return root
25 Function RangeSearch(root, [key1, key2]) do
26     value[]← ∅
27     left ← EqualitySearch(root, key1, l)
28     right ← EqualitySearch(root, key2, r)
       ▽ Traverse linked leaves in order
29     while left.next != right do
30         value.insert(left.next)
31         left ← left.next
32     return value
```

Second (**Proof checking**), on receiving the proof and the query result, the client ensures no manipulation or omission of query result by checking two rules. First, R_l's range is smaller than the left boundary's range, and R_r's range is larger than the right boundary's range. Second, the client recomputes the root subfilter in a bottom-up manner based on all the query result and all additional sibling subfilters, and compares the computed root subfilter with the one it owned. If two subfilters are the same, the query result is authenticated successfully.

Figure 5 shows a running example. Given a range query that searches for records in [K_1,K_2], it returns {N_{10}, N_{11}, N_{12}}

as the query result. For authentication, MFT returns $N_9 \sim N_{13}$: N_9 and N_{13} are returned to prove that no records in range $[K_1, N_{10})$ and $(N_{12}, K_2]$ are omitted. Two additional MFT nodes (N_4 and N_7) are also returned to reconstruct the root subfilter and compare to the local root subfilter for authentication.

Note that, we omit the discussion of authenticating equality queries here, because it takes the same theory of authentication and uses similar steps as range queries.

4.3 A Unified Query System Leveraging MFT

With MFT confidential index, HOTEE runs hybrid-trust multi-party joint query in three phases (Figure 2).

Phase 1: query dispatch. Before submitting queries, multiple parties first construct MFTs on their chosen index keys of data records (records are encrypted by either TEE using AES-GCM [32] or HE using BGV [24] based on independent trusts). Note that, MFT is constructed by attribute, multiple attributes can build multiple MFTs. Given the semi-honest assumption of clients (§3.2), a party cannot directly share its encryption keys for building MFTs with other parties, including the hash functions for generating subfilters (§4.1) and obfuscation parameters for generating biased ranges (§4.2). However, without knowing multiple parties' encryption keys, it is infeasible for a client (in one party) to construct query parameters (on index keys) that can be dispatched to query-dependent databases of multiple parties and then search in dispatched dependent databases.

To remedy this issue, our key observation is that even if multiple parties have distinct trusts of TEE or HE on protecting data records, query parameters are commonly less sensitive. Hence, it is secure to set up TEE enclaves (attested by all parties) at the proxy to store the encryption keys of parties, transform parameters and locate query-dependent databases without revealing multiple parties' encryption keys.

Specifically, a client sends a new query encrypted with AES to the proxy (❶), the proxy decrypts that query with AES decryption key in enclave, and transforms the parameters into rangefilters using each party's MFT encryption keys in an enclave (❷). By searching transformed rangefilters on MFTs, the proxy learns which databases contain the searched content of the query same as existing plaintext databases, while the proxy learns nothing about the plaintexts owing to the secure execution in an enclave. After searching, the proxy forms a party list P_{list} containing all related parties that the query depends on (for multi-party data aggregation), and dispatchs the transformed query to dependent databases.

Phase 2: local search. Upon receiving queries from the proxy, TEE databases decrypt query parameters with pre-stored keys in enclave and searches with a TEE-shielding B^+-tree where each tree node's value is plaintext; HE databases directly search on MFT with transformed parameters (❺).

Phase 3: data aggregate. Before sending searched query results (i.e., data records) to the client, a subtle case is that, since the records are retrieved from multiple parties, a client cannot decrypt all records because it knows only its party's decryption key. A trivial solution is to decrypt all retrieved records in the proxy enclave, and re-encrypt them with the client's pre-stored key, such that the client can decrypt data and the plaintext records are invisible to the semi-honest proxy. However, this solution does not work in a hybrid-trust scenario where records contain HE data, which cannot be decrypted in a TEE owing to the different trusts (§3.2).

We propose *SDA*, a protocol that securely aggregates multi-party HE data without decrypting thus violating HE's confidentiality, by leveraging two key weapons: the MFT confidential index and *key switch* [24], a homomorphic algorithm that can switch the encryption key of HE data to a new key without decrypting it. At a high level, SDA locates query-dependent parties by searching on MFT confidential index and produces P_{list} at the proxy in **Phase 1**, the proxy sends P_{list} to the client and the client requests *tokens* from parties in P_{list} (❸), finally the retrieved records are transformed by running key switch with *tokens* at the proxy and sent back to the client (❹).

Specifically, key switch (KS) consists of a tuple of algorithms KS = (CtxGen, Enc, Switch) with the following syntax:

- KS.CtxGen $\rightarrow ctx$. Generate a ciphertext ctx in the form of (a,A), in which $A = a \cdot S_A + m + te \bmod q$. S_A is the secret key owned by client A, m is the plaintext, a is a random parameter, other variants are modular parameters in HE.
- KS.Enc(S,S') $\rightarrow tkn$. Enc encrypts key S using a new key S' and outputs a token tkn. tkn takes the form of $(a^*, A^*) = (a', a' \cdot S' + S + te \bmod q)$, a' is a random parameter.
- KS.Switch(ctx,tkn) $\rightarrow ctx'$. By using tkn, Switch switches the encryption key of ctx from S to S' and outputs a new ciphertext $nctx = (-a \cdot a^*, A - a \cdot A^*)$.

Leveraging the above syntax, HOTEE's SDA protocol runs with three steps. First, HOTEE transforms query parameters and searches on MFTs at the proxy, locates databases that contain the matching data and generates a party list P_i that these databases belong to (§4.2). Second, the proxy signs P_i and returns P_i to the client, the client verifies the signature and sends i disposable encryption keys S_i to each party (❸). Upon receiving S_i, each party runs KS.Enc(S, S_i) to produce a secret token tkn_i by encrypting its encryption key with S_i, and sends tkn_i to the proxy enclave for aggregation later. Last, the proxy transforms each retrieved record from party P_i (i.e., ctx_i) with the party's token (i.e., tkn_i) by running KS.Switch(ctx_i, tkn_i) in enclave (❹), and sends transformed results to the client.

HOTEE emits three insights in **Phase 3**. First, the second step of requesting tokens in SDA protocol runs in parallel with **Phase 2** for efficient query processing. Second, by sending disposable new encryption keys to query-dependent parties to generate tokens and transforming query results with these tokens, the client can decrypt query results based on the new encryption keys without requiring multiple parties to directly share encryption keys, facilitating different parties to securely and jointly answer queries. Third, by storing tokens within the proxy enclave, HOTEE protects token confidentiality and

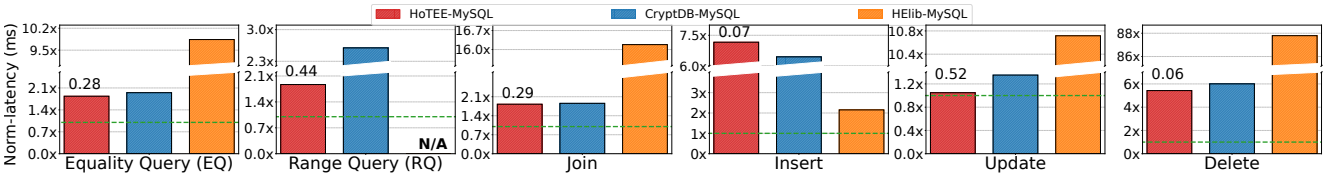


Figure 6: Normalized end-to-end latency of running TPC-C workload on all baselines. All systems' latency results are normalized to insecure MySQL query latency (green dashed line). The value on each red bar indicates HoTEE's averaged query latency (in milliseconds).

ensures that queries can only be transformed and decrypted by authenticated parties (clients).

4.4 Complexity and Security Analysis

Complexity analysis. Same as a traditional B$^+$-tree or other binary search trees, the time complexity of traversing inner nodes is $O(\log n)$, where n is the total number of MFT nodes; the time complexity of verifying subfilters at leaf nodes is $O(m)$, where m is the length of HoTEE's subfilter. Therefore, the time complexity of basic operations that traverse inner nodes and verify leaf nodes such as equality query, range query, insert, and delete is $O(\log n)+O(m)$. Since $m \ll n$, the actual time complexity for all basic operations is $O(\log n)$; the time complexity of other operations that combines basic operations including join and update is $O(\log n)$.

Security analysis. We give a two-step proof sketch of HoTEE's security guarantee by analyzing two core components created by HoTEE: the MFT index and SDA protocol.

• **Step 1: MFT index's confidentiality.** Same as existing EDBs [2, 42], we analyze the index confidentiality under chosen-plaintext attacks (CPA), a well-known attack that tries to infer plaintexts by using adversary-observable variables to deduce the black-box encryption algorithm.

Specifically, we illustrate MFT's confidentiality on data records and index keys separately. First, for data records, same as existing EDBs [42, 48], their confidentiality directly inherits from TEE's or HE's confidentiality guarantee, which is secure against CPA. Second, for the index keys on the database, a CPA attacker may use the observed rangefilter including subfilter and biased range to deduce plaintexts: for subfilters, it is impossible for the attacker to infer the plaintexts as we make standard assumptions on the one-wayness of cryptographic hash functions; for the biased range, even though the attacker may deduce the bias function owing to its linearity and monotonicity (formulated in §4.2), however, the attacker cannot infer the plaintext as well because the biased range is obfuscated with a client-chosen random parameter γ.

• **Step 2: SDA protocol's confidentiality.** The security of HoTEE's SDA protocol directly inherits the cryptographic guarantee of the key switch algorithm [24], a homomorphic algorithm that was used for reducing HE data's noises.

Overall, we prove that HoTEE provides the same strong confidentiality guarantee as prior work from the aspects of two core components: HoTEE's MFT index provides the same confidentiality against CPA attacks (**Step 1**) and the SDA protocol directly inherits the used HE algorithm's cryptographic guarantee (**Step 2**).

Distributed query latency (in milliseconds)

System (in SQL instance)	HoTEE		CryptDB	
Query type	**EQ**	**RQ**	**EQ**	**RQ**
P1: Parameter transform **(PR)**	0.07	0.06	0.2	0.35
P2: Global search **(PR)**	0.18	0.11	0.08	0.09
P3: Local search **(DB)**	4e-4	1e-3	2e-4	1e-3
P4: Query authenticate **(PR)**	1e-3	1e-3	N/A	N/A
P5: Key switch **(PR)**	0.1	0.22	0.05	0.18
P6: Result decrypt **(CL)**	0.15	0.42	0.21	0.52
End-to-end latency (**P1**+...+**P6**)	0.51	0.83	0.55	1.17

Table 3: End-to-end query latency breakdown. **EQ** is equality query, **RQ** is range query, **PR** is proxy, **DB** is database, **CL** is client.

5 Implementation

We implemented HoTEE and applied HoTEE to MySQL [39] and MongoDB's KV backend WiredTiger [16], with three optimizations (see details in Appendix A.1).

6 Evaluation

Testbed. All experiments were done on our cluster with 20 machines, each equipped with a 2.60GHz Intel E5-2690 V3 CPU with 24 cores, 64GB memory, and 40Gbps NIC. We ran each node (either a client, proxy, or database) in a docker container and used Linux tc [6] to control the RTT among any two nodes. The average ping latency was 0.17 ms.

Baseline. To evaluate the performance of HoTEE, we built a SQL (MySQL [39]) instance consisting of both HE nodes and TEE nodes, and another NoSQL (WiredTiger [16]) instance consisting of both HE nodes and TEE nodes. We compared two prevalent HE databases (i.e., CryptDB [42] and HElibDB [4]) and two native and insecure databases (i.e., MySQL and WiredTiger). MySQL is the most widely used SQL backend in recent EDBs; WiredTiger is the default NoSQL KV backend of MongoDB [16]. CryptDB uses deterministic encryption (i.e., DET [12]) for equality queries and order-preserving encryption (i.e., OPE [5]) for range queries by building indexes on DET data or OPE data separately. CryptDB supports general queries on HE data with logarithmic complexity; HElibDB is another prevalent KV-based HE database that utilizes specific homomorphic function (i.e., Fermat's little theorem [4]) to directly evaluate the equivalence of HE data by scanning all HE data linearly.

For an apple-to-apple comparison, we made three implementation efforts. First, we extended CryptDB's HE algorithm from partial HE to fully HE BGV [24] to have the same functionality as HoTEE. Second, all baseline systems were

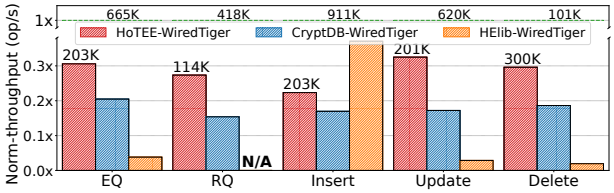
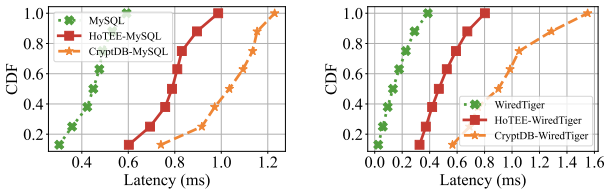


Figure 7: Normalized throughput of running all baselines (in NoSQL instance) locally. All throughput results are normalized to insecure WiredTiger's throughput (green dashed line, each value on the green dashed line is WiredTiger's averaged throughput). The value on each red bar indicates HoTEE's averaged throughput.



(a) Run in a SQL instance (b) Run in a NoSQL instance

Figure 8: Distributions of end-to-end range query latency running TPC-C and customized key-value workload on all baselines.

extensively implemented on both MySQL and WiredTiger. Third, we integrated HoTEE's core proxy components (see Figure 2) in CryptDB and HElibDB to make them support distributed queries same as HoTEE. All baselines were running with a state-of-the-art TEE database Azure EdgelessDB [2], which equips with an SGX-shielded B^+-tree for indexing.

Note that, we did not compare to other distributed encrypted databases because they either do not support query on HE data (e.g., HybrTC [54]) or focus on a specific type of document database (e.g., Dory [20]).

Workload. To the best of our knowledge, there is no standard benchmark for evaluating heterogeneous EDBs. Therefore, for a fair comparison, we evaluated workloads in all recent EDBs [2, 42, 55, 59], including the complex TPC-C benchmark [34] for relational databases and a simple customized workload for NoSQL key-value store.

Default setting. To match the multi-party deployment, we randomly partitioned the TPC-C across nodes based on warehouse id. Unless for the scalability experiment, we simulated 2 parties by default, one party runs a TEE node and the other party runs a HE node. Since a TPC-C client is already associated with a warehouse ID, we assigned each client to the party containing the client's warehouse. For NoSQL workload, we generated 2^{10} fixed-size keys (8 byte) and values (64 byte) same as [57], and horizontally partitioned all key-value pairs across nodes same as TPC-C. All workloads were encrypted. For the homomorphic setting, we used HElib's BGV implementation [4], and set the plaintext prime modulus as 29 and ciphertext modulus as 1009, to compress ciphertexts with an expansion coefficient of 4.

We ran each experiment for 60 seconds and collected the result in the middle 30s (i.e., 15s∼45s) to avoid the disturbance caused by system start-up and cool-down. Our evaluation focuses on the following questions:

§6.1 How efficient is HoTEE compared to baselines?

§6.2 Can HoTEE scale to more parties and larger datasets?

§6.3 Is HoTEE sensitive to the heterogeneity (ratio of TEE/HE nodes) on different queries?

§6.4 Is HoTEE robust to attacks?

6.1 End-to-end Performance

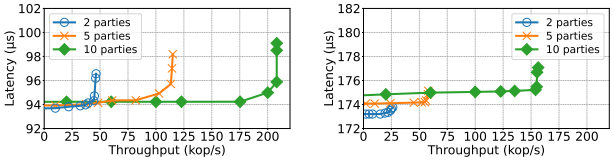
We first investigated the end-to-end performance of HoTEE and all baselines in the fault-free case (no manipulation or omission of query results). Figure 6 shows the comparison of all baselines running different types of SQL queries seen in TPC-C, where all systems are normalized to the insecure MySQL operations, which directly run on MySQL without TEE or HE protection. HoTEE's end-to-end latency (in milliseconds) is reported on its bar. N/A means a system cannot handle such a case because HElib uses homomorphic function (FLT [4]) that reports only the equivalence of HE data.

Favoring read query. HoTEE achieved 5.3X ∼ 16.1X lower latency (except for insert query) than HElib because HElib searches equivalent data through linear scans, whereas HoTEE searches on tree-based MFT indexes with logarithmic complexity, reducing the latency for all read queries.

Interestingly, HElib achieved 3X and 3.3X lower latency on insert queries than HoTEE and CryptDB, because HElib stores objects in append-only log files, whereas HoTEE and CryptDB both maintain sorted data in storage, thus incurring higher insert latency by first finding the position through indexing and then writing objects (§4.2). Favoring read performance is common in conventional databases [16, 39], while in HoTEE's multi-party joint query scenario, we insist the necessity of making such performance trade-off because read queries could occur far more frequently than write queries, hence we deem it acceptable to sacrifice moderate write performance to favor read performance.

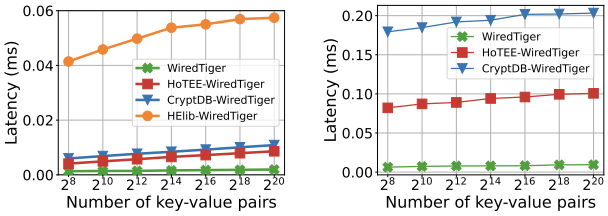
Efficiency on general queries. As shown in Figure 6, HoTEE achieved 6%∼28% lower query latency compared to CryptDB, which takes a cryptographic approach to encrypt data for equality queries with DET and encrypt data for range queries with OPE. By leveraging operation-specific encryption schemes, CryptDB also supports general queries with logarithmic search complexity, but HoTEE still outperformed CryptDB in terms of both the query latency (verified in Table 3) and throughput (Figure 7).

Specifically, from the latency breakdown shown in Table 3, we verified the efficiency that HoTEE's MFT index incurred less processing time than CryptDB's cryptographic approach. To dig into the latency speedup, we tested the representative range query (RQ) and evaluated its latency distributions, by running all baselines on general workloads and general databases, as depicted in Figrue 8: HoTEE significantly reduced all queries' latency with different range parameters compared to CryptDB, matching the result in Figure 6. The distributions was caused by different length-of-range of query parameters, a longer length typically leads to a larger number



(a) Equality Query (b) Range Query

Figure 9: HoTEE's throughput vs. latency when running with different numbers of parties.



(a) Equality Query (b) Range Query

Figure 10: Latency comparison of all baselines running key-value workload locally by increasing the size of key-value dataset.

of query results, bringing higher latency of key switch and decryption (except for insecure databases). Note that, HElib does not support range queries so we did not evaluate it.

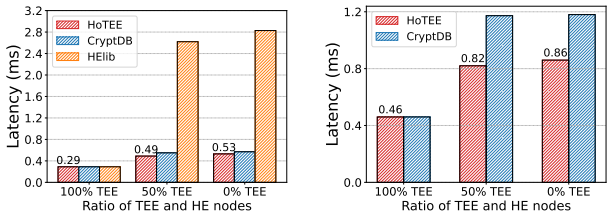
We evaluated all baselines' throughput by our simple key-value workload on a NoSQL instance (WiredTiger) locally and collected each operation's averaged throughput. As depicted in Figure 7, HoTEE achieved 1.31X∼1.88X higher throughput than CryptDB. This is because, when equality and range queries are mixed, CryptDB's design conducts frequent shuffling (re-encryption and decryption) between DET index and OPE index to support different queries; in contrast, HoTEE's MFT directly supports both equality and range queries in one single secure data structure thus avoids CryptDB's shuffling overhead (§4.2).

In summary, HoTEE is the only system that supports general query operations while still achieving lower end-to-end latency than all baselines for most query types. HoTEE favors read query performance and tolerate moderate write performance downgrading in the fault-free case, which we deemed acceptable in a multi-party joint query scenario.

6.2 Scalability

HoTEE is the first work that enables multiple parties to jointly answer queries using heterogeneous EDB nodes, thus HoTEE's scalability to the number of parties and the size of datasets is particularly important. In this subsection, we tested how HoTEE scales to more parties and larger datasets.

Scale to more parties. We let each party launch one TEE node and one HE node in a NoSQL instance, and we tested at most 10 parties because it is a sufficiently large number of parties for jointly queries in the real world [54]. Figure 9(a) and Figure 9(b) shows HoTEE's throughput-latency relationship with different numbers of parties running two types of queries. With more participating parties, both HoTEE's throughput and tail latency increased. Interestingly, the latency gap between different numbers of parties on range queries is larger



(a) Equality Query (b) Range Query

Figure 11: Heterogeneity study of running different ratio of TEE/HE nodes in a SQL instance on two typical queries.

than equality queries, because when the number of parties increases, a range query typically fetches more data than an equality query, leading to significantly increased processing time and a larger latency gap.

Scale to larger datasets. We present the latency comparison of all baselines running with different sizes of the key-value dataset locally. Figure 10(a) shows that, for equality queries, HoTEE and CryptDB both incurred moderate overhead than insecure WiredTiger, while HElibDB incurred much higher latency because it requires linearly scanning the whole dataset. Figure 10(b) shows that, for range queries, HoTEE incurred much lower latency than CryptDB (compared to equality queries) because HoTEE's MFT index took less time to transform queries compared to the cryptographic approach used by CryptDB (proved in Table 3), while HElibDB does not support range queries.

6.3 Sensitivity Study of Heterogeneity

We tested HoTEE's heterogeneity by varying the ratio of TEE/HE nodes in a SQL instance and running two major queries on every ratio respectively. Figure 11(a) shows that, a pure TEE node setting (i.e., 100% TEE) incurred much lower latency than other ratios. This indicates that, even if HoTEE achieves logarithmic search complexity and is much more efficient than all baselines, there still exists an efficiency gap between TEE and HE. This is because, TEE can directly search on decrypted data while searching on HE requires query transforming, searching and comparing MFT nodes, and key switching. Besides, compared to running range queries in Figure 11(b), the latency gap between a pure TEE (100% TEE nodes) setting and other ratios on equality queries in Figure 11(a) is much smaller, because executing an equality query is more efficient than executing a range query in HoTEE, which is also proved in Figure 9 and Figure 11.

6.4 Robustness to Attacks

The above evaluation focuses on a fault-free case where no manipulation or omission of query results occurs. It remains important for a client to authenticate queries by checking the absence and presence of data records in an outsourced database. However, all baselines ignore such authentication. In HoTEE, the MFT confidential index not only supports general queries with logarithmic complexity but also supports query authentication owing to the recursive Merkle-tree style

construction of MFT (§4.2). By checking the boundary conditions, re-constructing and comparing the root filter, HoTEE achieved 100% detection rate of any manipulation or omission of query results in our experiments.

6.5 Discussion and Limitation

Dismissing an alternative design. One may think of using only the monotonic biases as the keys to build an encrypted index [5], however, we insist the necessity of HoTEE's MFT from two aspects. First, a bias-only index uses only a linear function to produce index keys, which poses a higher confidentiality risk under CPA attacks, while MFT's rangefilter is more secure by relying on both cryptographic hash functions and confidential bias function with obfuscation (§4.2). Second, a bias-only index cannot authenticate queries, while MFT's recursive structure leverages irreversible rangefilter construction to check the manipulation or omission of data records in an outsourced database.

Limitations. HoTEE's design has two limitations. First, HoTEE leaks the order of ciphertexts on databases for comparison, which is an inherent issue with any encrypted databases that support range queries. Second, HoTEE's MFT index is built on a single searchable attribute, for databases that search over multiple attributes (e.g., graph databases [7, 31]), HoTEE handles only disjunctive queries by searching on MFTs of each attribute independently, however, for conjunctive queries, MFT has to be integrated with multi-dimentional indexes (e.g., k-d tree [58]), and we leave it for future work.

7 Conclusion

We present HoTEE, the first unified query system that can support general queries on heterogeneous EDB (i.e., simultaneously enabling HE and TEE nodes) from multiple parties with logarithmic search complexity, via a new MFT confidential index abstraction. By searching on MFTs and securely aggregating searched data, HoTEE creates a unified query system that bridges the silos between TEE and HE data, and facilitates multiple parties jointly answer queries using heterogeneous EDB nodes. Extensive results on both SQL and NoSQL instances show that HoTEE is general, extensible, and highly efficient compared to two notable EDBs. HoTEE is open source. HoTEE's source code is released on github.com/osdip281/HoTEE.

References

[1] Amazon AWS. 2022. State & local government. Retrieved 10/27/2022 from https://aws.amazon.com/stateandlocal/.

[2] Edgeless Systems GmbH. 2022. *EdgelessDB Official Website. Retrieved March 1, 2022*. https://www.edgeless.systems/products/edgelessdb.

[3] Google Cloud. 2022. Harvard global health institute covid-19 forecast. Retrieved 10/27/2022 from https://cloud.google.com/blog/products/ai-machine-learning/google-cloud-covid-19-public-forecasts.

[4] HElib-KV Lookup. 2022. *HElib Country Lookup Example.* https://github.com/homenc/HElib/tree/master/examples/BGV_country_db_lookup.

[5] Rakesh Agrawal, Jerry Kiernan, Ramakrishnan Srikant, and Yirong Xu. Order preserving encryption for numeric data. In *Proceedings of the 2004 ACM SIGMOD international conference on Management of data*, pages 563–574, 2004.

[6] Werner Almesberger et al. Linux network traffic control—implementation overview, 1999.

[7] Renzo Angles and Claudio Gutierrez. Survey of graph database models. *ACM Computing Surveys (CSUR)*, 40(1):1–39, 2008.

[8] Panagiotis Antonopoulos, Arvind Arasu, Kunal D Singh, Ken Eguro, Nitish Gupta, Rajat Jain, Raghav Kaushik, Hanuma Kodavalla, Donald Kossmann, Nikolas Ogg, et al. Azure sql database always encrypted. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 1511–1525, 2020.

[9] Arvind Arasu, Spyros Blanas, Ken Eguro, Manas Joglekar, Raghav Kaushik, Donald Kossmann, Ravi Ramamurthy, Prasang Upadhyaya, and Ramarathnam Venkatesan. Secure database-as-a-service with cipherbase. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*, pages 1033–1036, 2013.

[10] Arvind Arasu, Spyros Blanas, Ken Eguro, Raghav Kaushik, Donald Kossmann, Ravishankar Ramamurthy, and Ramarathnam Venkatesan. Orthogonal security with cipherbase. In *CIDR*, 2013.

[11] David W Archer, Dan Bogdanov, Yehuda Lindell, Liina Kamm, Kurt Nielsen, Jakob Illeborg Pagter, Nigel P Smart, and Rebecca N Wright. From keys to databases—real-world applications of secure multiparty computation. *The Computer Journal*, 61(12):1749–1771, 2018.

[12] Mihir Bellare, Marc Fischlin, Adam O'Neill, and Thomas Ristenpart. Deterministic encryption: Definitional equivalences and constructions without random oracles. In *Annual International Cryptology Conference*, pages 360–378. Springer, 2008.

[13] Burton H Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426, 1970.

[14] Marcus Brandenburger, Christian Cachin, Matthias Lorenz, and Rüdiger Kapitza. Rollback and forking detection for trusted execution environments using lightweight collective memory. In *2017 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 157–168. IEEE, 2017.

[15] Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C Hsieh, Deborah A Wallach, Mike Burrows, Tushar Chandra, Andrew Fikes, and Robert E Gruber. Bigtable: A distributed storage system for structured data. *ACM Transactions on Computer Systems (TOCS)*, 26(2):1–26, 2008.

[16] Rupali Chopade and Vinod Pachghare. Mongodb indexing for performance improvement. In *ICT Systems and Sustainability*, pages 529–539. Springer, 2020.

[17] Victor Costan and Srinivas Devadas. Intel sgx explained. *Cryptology ePrint Archive*, 2016.

[18] Ronald Cramer, Ivan Bjerre Damgård, et al. *Secure multiparty computation*. Cambridge University Press, 2015.

[19] Emma Dauterman, Vivian Fang, Ioannis Demertzis, Natacha Crooks, and Raluca Ada Popa. Snoopy: Surpassing the scalability bottleneck of oblivious storage. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, pages 655–671, 2021.

[20] Emma Dauterman, Eric Feng, Ellen Luo, Raluca Ada Popa, and Ion Stoica. Dory: An encrypted search system with distributed trust. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 1101–1119, 2020.

[21] Yarkın Doröz, Gizem S Çetin, and Berk Sunar. On-the-fly homomorphic batching/unbatching. In *International Conference on Financial Cryptography and Data Security*, pages 288–301. Springer, 2016.

[22] Saba Eskandarian and Matei Zaharia. Oblidb: Oblivious query processing for secure databases. *arXiv preprint arXiv:1710.00458*, 2017.

[23] ByteDance Jeddak for privacy computing and federated learning. 2022. Jeddak project. Retrieved 10/27/2022 from https://github.com/bytedance-jeddak/jeddak.

[24] Craig Gentry, Shai Halevi, Chris Peikert, and Nigel P Smart. Ring switching in bgv-style homomorphic encryption. In *International Conference on Security and Cryptography for Networks*, pages 19–37. Springer, 2012.

[25] Vipul Goyal, Adam O'Neill, and Vanishree Rao. Correlated-input secure hash functions. In *Theory of Cryptography Conference*, pages 182–200. Springer, 2011.

[26] Paul Grubbs, Thomas Ristenpart, and Vitaly Shmatikov. Why your encrypted database is not secure. In *Proceedings of the 16th workshop on hot topics in operating systems*, pages 162–168, 2017.

[27] Shai Halevi and Victor Shoup. Algorithms in helib. In *Annual Cryptology Conference*, pages 554–571. Springer, 2014.

[28] James L Hieronymus. Ascii phonetic symbols for the world's languages: Worldbet. *Journal of the International Phonetic Association*, 23:72, 1993.

[29] Ilia Iliashenko and Vincent Zucca. Faster homomorphic comparison operations for bgv and bfv. *Proceedings on Privacy Enhancing Technologies*, 2021(3):246–264, 2021.

[30] Scott R. Intel. *SGX 2.0 (scalable SGX).* https://github.com/intel/linux-sgx/issues/899.

[31] Borislav Iordanov. Hypergraphdb: a generalized graph database. In *International conference on web-age information management*, pages 25–36. Springer, 2010.

[32] Emilia Käsper and Peter Schwabe. Faster and timing-attack resistant aes-gcm. In *International Workshop on Cryptographic Hardware and Embedded Systems*, pages 1–17. Springer, 2009.

[33] Brian Knott, Shobha Venkataraman, Awni Hannun, Shubho Sengupta, Mark Ibrahim, and Laurens van der Maaten. Crypten: Secure multi-party computation meets machine learning. *Advances in Neural Information Processing Systems*, 34:4961–4973, 2021.

[34] Scott T Leutenegger and Daniel Dias. A modeling study of the tpc-c benchmark. *ACM Sigmod Record*, 22(2):22–31, 1993.

[35] Dong Li, Xiaofeng Liao, Tao Xiang, Jiahui Wu, and Junqing Le. Privacy-preserving self-serviced medical diagnosis scheme based on secure multi-party computation. *Computers & Security*, 90:101701, 2020.

[36] Dongxi Liu and Shenlu Wang. Programmable order-preserving secure index for encrypted database query. In *2012 IEEE Fifth International Conference on Cloud Computing*, pages 502–509. IEEE, 2012.

[37] Yanbin Lu et al. Privacy-preserving logarithmic-time search on encrypted data in cloud. In *NDSS*, 2012.

[38] Murali Mani, Kinnari Shah, and Manikanta Gunda. Enabling secure database as a service using fully homomorphic encryption: Challenges and opportunities. *arXiv preprint arXiv:1302.2654*, 2013.

[39] AB MySQL. Mysql, 2001.

[40] Proteet Paul, Tushar Gupta, and Shamik Sural. Poster: Asql-attribute based access control extension for sql. In *Proceedings of the 27th ACM on Symposium on Access Control Models and Technologies*, pages 259–261, 2022.

[41] Rishabh Poddar, Tobias Boelter, and Raluca Ada Popa. Arx: A strongly encrypted database system. *IACR Cryptol. ePrint Arch.*, 2016:591, 2016.

[42] Raluca Ada Popa, Catherine MS Redfield, Nickolai Zeldovich, and Hari Balakrishnan. Cryptdb: protecting confidentiality with encrypted query processing. In *Proceedings of the twenty-third ACM symposium on operating systems principles*, pages 85–100, 2011.

[43] Behandelt PostgreSQL. Postgresql. *Web resource: http://www. PostgreSQL. org/about*, 1996.

[44] Christian Priebe, Kapil Vaswani, and Manuel Costa. Enclavedb: A secure database using sgx. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 264–278. IEEE, 2018.

[45] Addanki Purna Ramesh, AVN Tilak, and AM Prasad. An fpga based high speed ieee-754 double precision floating point multiplier using verilog. In *2013 International Conference on Emerging Trends in VLSI, Embedded System, Nano Electronics and Telecommunication System (ICEVENT)*, pages 1–5. IEEE, 2013.

[46] Mario Schkolnick. Secondary index optimization. In *Proceedings of the 1975 ACM SIGMOD international conference on Management of data*, pages 186–192, 1975.

[47] Erez Shmueli, Ronen Vaisenberg, Ehud Gudes, and Yuval Elovici. Implementing a database encryption solution, design and implementation issues. *Computers & security*, 44:33–50, 2014.

[48] Yuanyuan Sun, Sheng Wang, Huorong Li, and Feifei Li. Building enclave-native storage engines for practical encrypted databases. *Proceedings of the VLDB Endowment*, 14(6):1019–1032, 2021.

[49] Chia-Che Tsai, Donald E Porter, and Mona Vij. {Graphene-SGX}: A practical library {OS} for unmodified applications on {SGX}. In *2017 USENIX Annual Technical Conference (USENIX ATC 17)*, pages 645–658, 2017.

[50] Eugene Tsyrklevich and Bennet Yee. Dynamic detection and prevention of race conditions in file accesses. In *12th USENIX Security Symposium (USENIX Security 03)*, 2003.

[51] Stephen Lyle Tu, M Frans Kaashoek, Samuel R Madden, and Nickolai Zeldovich. Processing analytical queries over encrypted data. 2013.

[52] Harshavardhan Unnibhavi, David Cerdeira, Antonio Barbalace, Nuno Santos, and Pramod Bhatotia. Secure and policy-compliant query processing on heterogeneous computational storage architectures. In *ACM SIGMOD/PODS International Conference on Management of Data 2022*, 2022.

[53] Xingwei Wang, Hong Zhao, and Jiakeng Zhu. Grpc: A communication cooperation mechanism in distributed systems. *ACM SIGOPS Operating Systems Review*, 27(3):75–86, 1993.

[54] Pengfei Wu, Jianting Ning, Jiamin Shen, Hongbing Wang, and Ee-Chien Chang. Hybrid trust multi-party computation with trusted execution environment.

[55] Yu Xia, Xiangyao Yu, Matthew Butrovich, Andrew Pavlo, and Srinivas Devadas. Litmus: Towards a practical database management system with verifiable acid properties and transaction correctness. In *Proceedings of the 2022 International Conference on Management of Data, Philadelphia, PA, USA*, pages 12–17, 2022.

[56] Statistics Canada. Zachary Zanussi. *Privacy Preserving Technologies Part Two: Introduction to Homomorphic Encryption.* https://www.statcan.gc.ca/en/about/statcan#a7.

[57] Yuhong Zhong, Haoyu Li, Yu Jian Wu, Ioannis Zarkadas, Jeffrey Tao, Evan Mesterhazy, Michael Makris, Junfeng Yang, Amy Tai, Ryan Stutsman, et al. {XRP}:{In-Kernel} storage functions with {eBPF}. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 375–393, 2022.

[58] Kun Zhou, Qiming Hou, Rui Wang, and Baining Guo. Real-time kd-tree construction on graphics hardware. *ACM Transactions on Graphics (TOG)*, 27(5):1–11, 2008.

[59] Wenchao Zhou, Yifan Cai, Yanqing Peng, Sheng Wang, Ke Ma, and Feifei Li. Veridb: An sgx-based verifiable database. In *Proceedings of the 2021 International Conference on Management of Data*, pages 2182–2194, 2021.

A Appendix

A.1 Implementation Details

We implemented HoTEE with 5021 lines of C++ code on the CryptDB codebase [42], a modular framework for evaluating distributed encrypted databases. All of HoTEE's protocol messages (❶, ❸) were implemented with asynchronous RPC calls [53], and we instantiated messages using AES [32]. We modified CryptDB proxy with around 200 lines of Graphene [49] code for enclave instantiation and code wrapping (in `pr.manifest`), and spawned two enclave threads: one (`ts_thread`) for profiling, transforming queries, and searching on MFT indexes to locate query-dependent databases (❷); the other one (`ds_thread`) runs SDA protocol to generate and fetch key switch tokens (§4.3), and aggregates multi-party encrypted data with the tokens (❹).

Case study. To understand whether HoTEE can benefit real world databases, we built two HE databases on MySQL [39] and MongoDB's default KV backend WiredTiger [16], by replacing MySQL's default B$^+$-tree and WiredTiger's B-tree with MFT confidential index. Especially, since WiredTiger maintains an LRU cache for queries' entire B-tree traversal path including both inner and leaf nodes' pages, to comply with such caching semantics, HoTEE's search operations described in the previous section (❺) also returns all traversed MFT pages so that WiredTiger can cache them.

Optimization. First, we took a batching strategy [21] on homomorphic encryption/decryption and key switching to maximize the throughput of HE databases. Second, we fine-tuned the WiredTiger's LRU cache size to maximize cache hit ratio while keeping a large fraction of the available memory for MFT indexes for fast data retrieval. Last, we constructed MFT's rangefilter nodes by storing only the `"1"` bits and skipping the `"0"` bits to keep larger working sets of MFT indexes in memory and reduce time-consuming disk I/O.

A.2 Generality of HoTEE

Is HoTEE general? We illustrate HoTEE's generality by discussing two HoTEE's core components: MFT confidential index and SDA protocol. First, MFT is general to database types including both relational and NoSQL databases, because we assume a widely-adopted unclustered architecture where index and other components (e.g., storage) are decoupled (§3.1) and MFT only substitutes the original index; MFT is also general to attribute types (integer, real value, and string): as MFT so far works on the integer attribute, we can transform real values into integers using IEEE 754 floating point number [45] and transform string values using ASCII character [28] by encoding any length-l ASCII string with integers $[0, 2^{7 \cdot l} - 1]$. Second, SDA is general as well, because SDA uses the generic MFT for searching, and aggregates data with key switch, a generic tool in common HE algorithms to reduce HE data's cumulative noises [24, 29].